1

HỌC MÁY
TĂNG CƯỜNG CHO
CÁC HỆ THỐNG MẠNG

Làm quen với Python, Gymnasium và xây dựng bài toán RL

Biên soạn: ThS. Phan Trung Phát
Liên hệ: phatpt@uit.edu.vn

Tái bản lần 1 - Tháng 03/2026

Lưu hành nội bộ

A. MỤC TIÊU

- Giới thiệu với ngôn ngữ lập trình Python.
- Làm quen công cụ Gymnasium thằng tạo môi trường trong Học tăng cường (Reinforcement Learning, RL).
- Giới thiệu bài toán ứng dụng Load Balancing trong hệ thống.
- Phân tích và hiện thực hoá bài toán theo Makov Decision Process (MDP).
- Thực hiện đánh giá và điểm thử trên các chính sách.

B. GIỚI THIỆU

1. Ngôn ngữ lập trình Python

a. Sự phù hợp và điểm mạnh

Python được khởi xướng bởi Guido van Rossum vào cuối những năm 1980 tại Hà Lan và chính thức ra mắt phiên bản 1.0 vào năm 1994. Triết lý thiết kế của Python được tóm tắt qua khẩu hiệu "Beautiful is better than ugly" (Cái đẹp tốt hơn cái xấu), nhấn mạnh vào sự đơn giản, rõ ràng và khả năng đọc hiểu mã nguồn tối ưu. Khác với C++ được phát triển từ nền tảng lập trình hệ thống, Python được định vị là một ngôn ngữ lập trình bậc cao, đa mục tiêu và hỗ trợ nhiều phong cách lập trình (hướng đối tượng, lập trình chức năng, lập trình cấu trúc).

Python không chỉ là một công cụ lập trình, mà là một hệ sinh thái (ecosystem) đóng vai trò hạt nhân trong cuộc cách mạng trí tuệ nhân tạo ở thời điểm hiện tại. Một số đặc tính kỹ thuật mấu chốt có thể kể đến như:

- **Triết lý thiết kế tối giản**: Sử dụng cấu trúc thụt lề (indentation) thay vì các khối lệnh rườm rà, tập trung vào tối ưu hoá năng suất của người lập trình. Đồng thời, bỏ các ký hiệu cú pháp phức tạp, giúp mã nguồn trở nên sạch sẽ, giống như ngôn ngữ tự nhiên.

- **Phù hợp với tư duy Toán học**: Các thuật toán RL dựa trên nền tảng xác suất và tối ưu hóa. Python cho phép chuyển đổi gần như 1:1 từ công thức toán học sang mã code, giúp dễ dàng theo dõi và thực hành.

Sức mạnh của Python không chỉ nằm ở cốt lõi ngôn ngữ mà còn ở khả năng mở rộng và tích hợp. Đối với môn học này, 3 trụ cột thư viện quan trọng tương thích lớn với Python gồm:

1. **Tính toán khoa học** (NumPy[1]): Xử lý các ma trận xác suất chuyển trạng thái, hỗ trợ tính toán số học trong các thuật toán RL.
2. **Mô phỏng môi trường** (Gymnasium[2]): Cung cấp giao diện chuẩn để kết nối thuật toán RL với các bài toán thực tế bằng cách mô hình hoá, mô phỏng vận hành.
3. **Học sâu** (PyTorch[3]/TensorFlow[4]): Công cụ để triển khai các mạng thần kinh (neural networks) đóng vai trò là bộ não của các tác nhân *Học tăng cường sâu* và quá trình đào tạo mô hình.

Mặc dù là ngôn ngữ ưu việt để phát triển AI, Python vẫn tồn tại những điểm yếu về mặt kỹ thuật. Vấn đề lớn nhất là tốc độ thực thi chậm do bản chất thông dịch và cơ chế quản lý bộ nhớ tự động gây tốn tài nguyên RAM, khiến nó không phù hợp cho các tác vụ xử lý gói tin ở thời gian thực. Thêm vào đó, cơ chế khóa GIL tạo ra rào cản khi cần tính toán song song quy mô lớn (vẫn giải quyết được với các thư viện hỗ trợ). Tuy nhiên trong môn học này, chúng ta có thể chấp nhận những hạn chế đó để đổi lấy năng suất lập trình vượt trội mà không bị sa lầy vào quá trình tối ưu hoá hiệu suất của hệ thống đối với ngôn ngữ lập trình - vốn không nằm trong giới hạn của môn học.

[1] https://numpy.org/
[2] https://gymnasium.farama.org/
[3] https://pytorch.org/
[4] https://www.tensorflow.org/

b. Những khác biệt mấu chốt về cú pháp

Điểm khác biệt đầu tiên trong Python, bắt đầu từ **thụt lề** và là cách duy nhất để định nghĩa cấu trúc chương trình (vòng lặp, hàm, câu lệnh điều kiện). Đồng thời, Python sử dụng ký tự xuống dòng để kết thúc một câu lệnh. Dù bạn có thể dùng `;` để viết nhiều lệnh trên một dòng, nhưng điều này bị coi là không đúng phong cách (un-Pythonic).

```
# Hello World function
#
def main():
    hello_world = "Hello, World!"
    print(hello_world)

if __name__ == "__main__":
    main()
```

Tiếp theo, Python xác định kiểu dữ liệu tại thời điểm chạy. Ví dụ chỉ cần viết `a = 10`. Python tự hiểu `a` là số nguyên. Nếu dòng sau bạn viết `a = "Hello"`, Python sẽ tự chuyển `a` thành kiểu chuỗi mà không báo lỗi.

Cú pháp các **câu lệnh điều kiện** và **vòng lặp** trong Python được giản lược tối đa để gần với ngôn ngữ tự nhiên:

```
// Hello World function
//
x = 5
if x > 0 and x < 10:
    print("Valid")
else:
    print("Invalid")
```

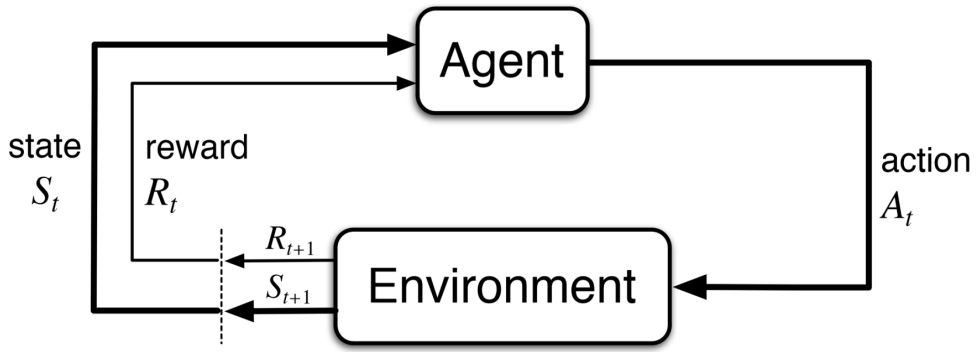
```
// Loop
//
for i in range(5):
    print(f"Iteration {i}")
print("Loop finished!")
```

Python lặp dựa trên một tập hợp (collection) hoặc một khoảng giá trị (range), giúp mã nguồn ngắn gọn hơn nhiều.

Ngoài ra, còn rất nhiều sự khác biệt trong cú pháp lập trình của Python như: biến, xử lý cấu trúc dữ liệu, hướng đối tượng trong Python,..., tuy nhiên có thể dễ dàng học được[5] do tính phổ dụng của nó. Đối với phần xử lý dữ liệu chuyên sâu, sinh viên có thể tìm hiểu thêm thư viện **Numpy**[1] để thực hiện việc này đơn giản hơn.

2. Gymnasium

Trong lĩnh vực Học máy tăng cường, việc tạo ra một môi trường giả lập để "Tác nhân" (Agent) có thể học hỏi thông qua thử và sai là bước đi tiên quyết. **Gymnasium** chính là bộ thư viện tiêu chuẩn thế giới hiện nay để thực hiện nhiệm vụ đó.



Hình 1. Vòng lặp của học tăng cường

Hành trình của **Gymnasium**[2] bắt đầu từ **OpenAI Gym**, một thư viện được phát triển bởi OpenAI vào năm 2016 với mục đích chuẩn hóa cách thức các nhà nghiên cứu RL tương tác với môi trường mô phỏng. Trước đó, mỗi nhà nghiên cứu thường tự viết môi trường riêng, khiến việc so sánh hiệu quả giữa các thuật toán trở nên rất khó khăn.

Tuy nhiên, vào khoảng năm 2021, OpenAI quyết định chuyển trọng tâm sang các dự án khác. Để duy trì và tiếp tục phát triển bộ công cụ quan trọng này, cộng đồng mã nguồn mở đã tiếp nhận và duy trì nó dưới một dự án mới có tên là Gymnasium, được quản lý bởi Farama

[5] https://www.python.org/

Foundation. Gymnasium kế thừa toàn bộ tinh hoa của OpenAI Gym nhưng được tối ưu hóa về hiệu năng, sửa các lỗi tồn đọng và tương thích tốt hơn với các phiên bản Python hiện đại.

Gymnasium hoạt động dựa trên mô hình tương tác kinh điển của RL: Agent - Environment Loop. Nó không chỉ cung cấp các trò chơi sẵn có mà là một framework mạnh mẽ cho phép bạn làm được những điều sau:

- **Chuẩn hóa giao diện**: Gymnasium ép buộc mọi môi trường phải tuân thủ một vòng đời nghiêm ngặt thông qua các hàm cốt lõi, ví dụ như `reset()` (để khởi động lại Environment) và `step(action)` (để thực hiện một hành động).
- **Không gian Trạng thái và Hành động** (*Spaces*): Gymnasium định nghĩa rõ ràng các kiểu dữ liệu cho Environment thông qua `Box` (giá trị liên tục - ví dụ: điện áp, băng thông) và `Discrete` (giá trị rời rạc - ví dụ: chọn Server A hoặc Server B).
- **Cơ chế Wrapper**: Cho phép lập trình viên dễ dàng chỉnh sửa dữ liệu đầu vào hoặc đầu ra của Environment mà không cần can thiệp sâu vào code lõi.
- **Khả năng mô phỏng và mở rộng môi trường tuỳ chỉnh**: Tích hợp sẵn các công cụ như `render()` để trực quan hóa quá trình học (có thể bằng giao diện chẳng hạn). Ngoài các Environment được cung cấp sẵn, Gymnasium còn cho phép người dùng tự định nghĩa môi trường riêng (Custom Environment[6]).

3. Bài toán Load Balancing

Trong thiết kế hệ thống (System Design), **Load Balancing** được hiểu là quá trình phân phối lưu lượng truy cập một cách chiến lược trên nhiều máy chủ khác nhau nhằm đảm bảo không có bất kỳ một máy chủ đơn lẻ nào rơi vào tình trạng quá tải (overload).

Hãy tưởng tượng một hệ thống mạng giống như một siêu thị: Nếu tất cả khách hàng đều dồn vào một quầy thu ngân duy nhất, sự tắc nghẽn là không thể tránh khỏi dù nhân viên đó

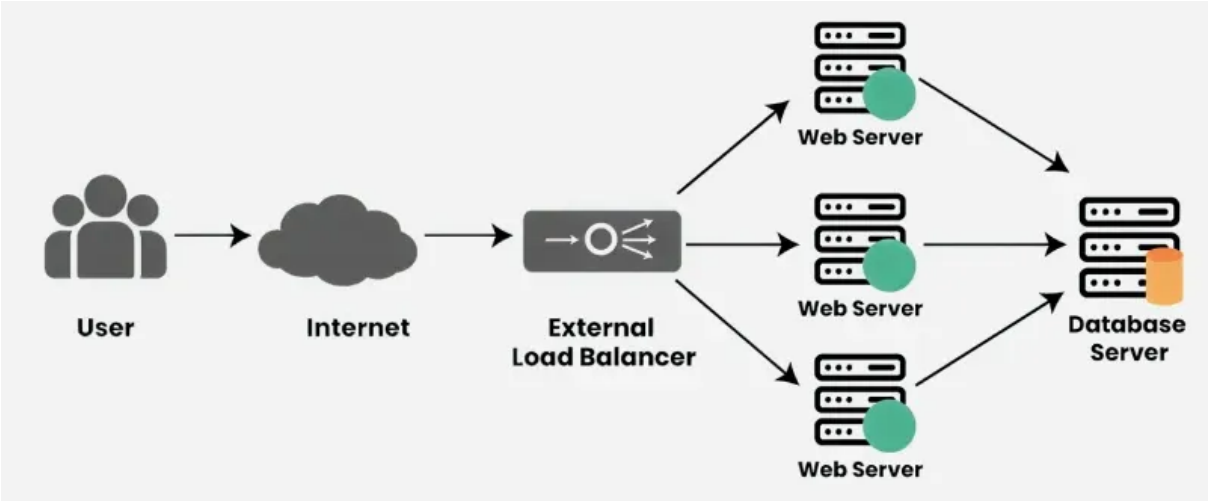
[6] https://gymnasium.farama.org/introduction/create_custom_env/

có làm việc nhanh đến đâu. Load Balancer đóng vai trò như một người điều phối tại sảnh, hướng dẫn khách hàng đến các quầy đang trống hoặc có ít người đợi nhất, từ đó tối ưu hóa toàn bộ quá trình phục vụ.

Load Balancing có khả năng giải quyết ba thách thức lớn của hệ thống mạng hiện đại[7]:

- **Tính khả dụng cao** (*Availability*): Trong thực tế, máy chủ có thể bị lỗi do phần cứng hoặc cần bảo trì phần mềm. Load Balancer giúp hệ thống duy trì hoạt động liên tục bằng cách tự động phát hiện các máy chủ bị lỗi và chuyển hướng lưu lượng sang các máy chủ đang hoạt động bình thường. Điều này giúp loại bỏ "điểm lỗi duy nhất" (Single Point of Failure).
- **Khả năng mở rộng** (*Scalability*): Khi nhu cầu người dùng tăng cao, thay vì phải nâng cấp một chiếc máy chủ cũ lên cấu hình cực mạnh (Vertical Scaling - tốn kém và có giới hạn), chúng ta có thể đơn giản là thêm nhiều máy chủ bình thường vào hệ thống (Horizontal Scaling). Load Balancer sẽ tự động nhận diện và chia tải cho các thành viên mới này.
- **Tối ưu hóa hiệu suất** (*Performance*): Bằng cách phân tán tải đều, mỗi yêu cầu của người dùng được xử lý trong thời gian ngắn nhất có thể, giảm thiểu độ trễ (latency) và tránh hiện tượng "thắt nút cổ chai" tại một node mạng cụ thể.

[7] https://techdocs.akamai.com/cloud-computing/docs/introduction-to-load-balancing



Hình 2. Vị trí triển khai Load Balancer trong hệ thống

Thông thường, Load Balancer được đặt giữa thiết bị của người dùng cuối (Client) và các máy chủ cung cấp dịch vụ (Backend Servers). Nó hoạt động như một "người đại diện" (Reverse Proxy) tiếp nhận tất cả các yêu cầu đến, sau đó sử dụng các thuật toán điều phối để đưa ra quyết định chuyển tiếp dữ liệu đến máy chủ phù hợp nhất[8].

Hiện nay, trong môi trường điện toán đám mây (điển hình như AWS hay Microsoft Azure) đều tích hợp Load Balancing sâu vào hạ tầng để xử lý hàng tỷ yêu cầu mỗi giây với độ tin cậy gần như tuyệt đối. Không những thế, nó còn được cung cấp như một dịch vụ (Load Balancing as a Service - LBaaS) cho hàng triệu doanh nghiệp.

Các thuật toán điều phối thông thường cho các Load Balancer đang được sử dụng có thể kể đến như[9,10]:

1. **Round Robin** (*Xoay vòng*): Đây là phương pháp cơ bản nhất, trong đó các yêu cầu được gửi lần lượt đến từng máy chủ theo một chu kỳ lặp lại. Thuật toán này hiệu

[8] https://www.geeksforgeeks.org/system-design/what-is-load-balancer-system-design/
[9] https://aws.amazon.com/what-is/load-balancing/#how-can-aws-help-with-load-balancing--1v2lmja
[10] https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/load-balancing-overview?toc=%2Fazure%2Fload-balancer%2Ftoc.json

quả khi các máy chủ có cấu hình tương đương và các tác vụ có độ nặng nhẹ giống nhau.

2. **Weighted Round Robin** (*Xoay vòng có trọng số*): Cải tiến từ Round Robin, thuật toán này gán cho mỗi máy chủ một "trọng số" dựa trên năng lực xử lý (CPU/RAM). Máy chủ mạnh hơn sẽ nhận được tỷ lệ yêu cầu cao hơn.
3. **Least Connections** (*Kết nối ít nhất*): Load Balancer sẽ ưu tiên đẩy lưu lượng vào máy chủ hiện đang có ít phiên làm việc (session) đang hoạt động nhất. Đây là giải pháp cực kỳ hiệu quả cho các ứng dụng có thời gian kết nối dài (như streaming hoặc chơi game).
4. **Priority-based** (*Dựa trên mức ưu tiên*): Hệ thống phân chia các máy chủ thành các nhóm ưu tiên khác nhau. Nhóm có ưu tiên thấp (Backup) chỉ được sử dụng khi nhóm ưu tiên cao nhất bị quá tải hoặc gặp sự cố.

C. THỰC HÀNH

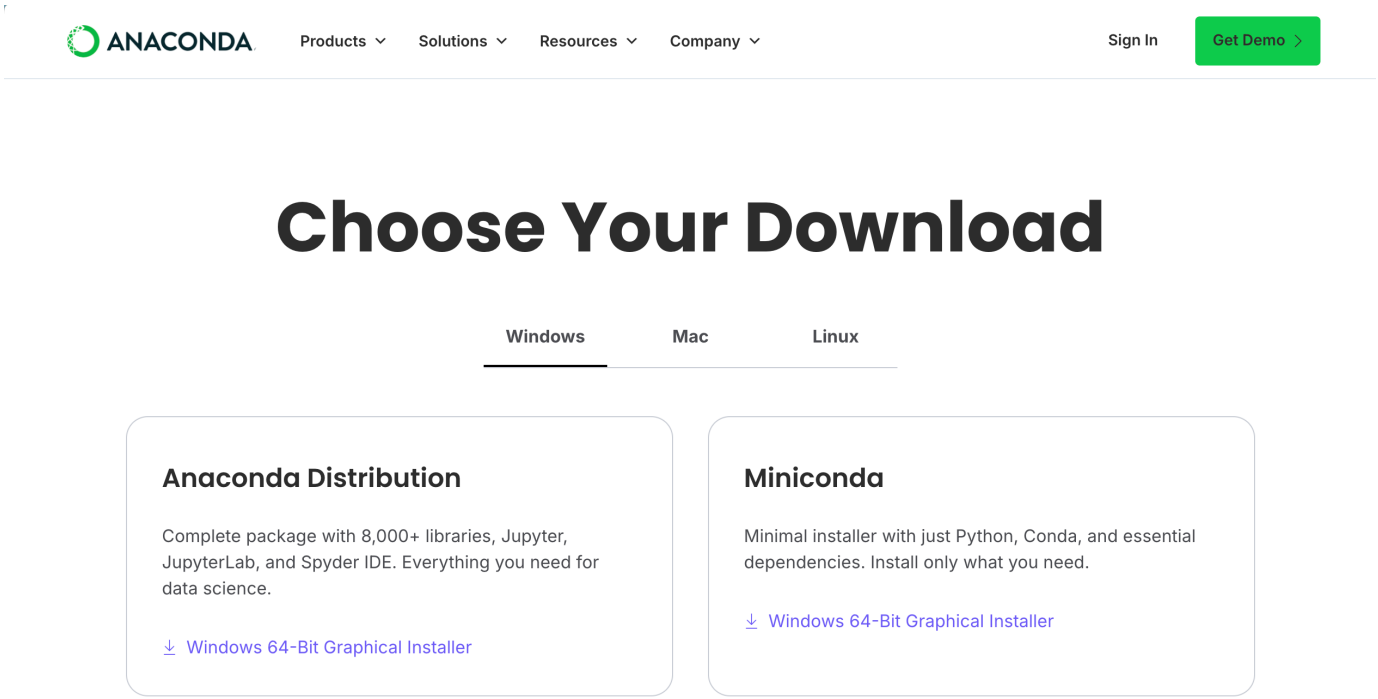
1. Lập trình với Python

a. Cài đặt Python bằng Conda trên Visual Studio Code

Mô tả: Tại phần thực hành này, sinh viên tiến hành cài đặt Conda để tạo môi trường ảo, Python sẽ được cài lên môi trường này. Thực hiện lập trình, tìm hiểu và làm quen với ngôn ngữ lập trình Python.

- **Bước 1:** Tải và cài đặt Miniconda hoặc Anaconda:

Conda là trình quản lý môi trường mạnh mẽ nhất cho AI, giúp bạn cô lập các thư viện để chúng không xung đột lẫn nhau. Nếu máy tính của sinh viên có dung lượng thấp, hãy cài **Miniconda**. Nếu muốn có sẵn mọi thứ, hãy cài **Anaconda**.



- **Bước 2:** Sau khi đã cài đặt xong, mở **Terminal** (hoặc Anaconda Prompt) và kiểm tra bằng lệnh: `conda --version` . Có thể thêm Miniconda3 vào trong *PATH environment variable* của Windows để tiện gọi lại sau này.

```
phantrungphat@Phats-MacBook-Pro vscode_workspace % conda --version
conda 23.5.0
phantrungphat@Phats-MacBook-Pro vscode_workspace %
```

- **Bước 3:** Tạo phân vùng riêng cho mã nguồn Python cần chạy.

  Sử dụng lệnh: `conda activate` để kích hoạt môi trường *(base)* mặc định.

  Sử dụng lệnh: `conda create -n nt549-x-name python=3.12` để tạo môi trường với tên gọi *nt549-x-name.* Trong đó *x* là số thứ tự nhóm và *name* là tên của cá nhân.

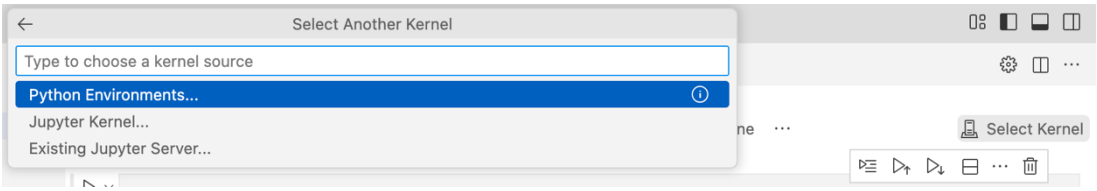
```
phantrungphat@Phats-MacBook-Pro vscode_workspace % conda activate
(base) phantrungphat@Phats-MacBook-Pro vscode_workspace % conda create -n nt549-1-phatpt python=3.12
Retrieving notices: ...working... done
Collecting package metadata (current_repodata.json): done
Solving environment: done
```

- **Bước 4:** Chuyển sang môi trường mới tạo bằng lệnh `conda activate nt549-x-name`

  Kiểm tra lại Python sau khi đã cài đặt bằng Conda.

```
(base) phantrungphat@Phats-MacBook-Pro vscode_workspace % conda activate nt549-1-phatpt
(nt549-1-phatpt) phantrungphat@Phats-MacBook-Pro vscode_workspace % python3 --version
Python 3.12.11
(nt549-1-phatpt) phantrungphat@Phats-MacBook-Pro vscode_workspace %
```

- **Bước 5:** Cài đặt thư viện hỗ trợ Notebook: `conda install ipykernel`
- **Bước 6:** Mở Visual Studio Code, vào mục **Extensions**, tìm và cài đặt 2 gói: **Python** và **Jupyter**.
- **Bước 7:** Mở file **Ex1.ipynb** được cung cấp tại đây.
- **Bước 8:** Ở góc trên bên phải của file, chọn **Select Kernel → Python Environments...** và chọn vào tên môi trường đã tạo ở trên.



- **Bước 9:** Chọn **Run All** hoặc chạy từng khối lệnh được cung cấp sẵn trong file và quan sát kết quả thực thi các đoạn mã Python.

2. Lập trình với Gymnasium

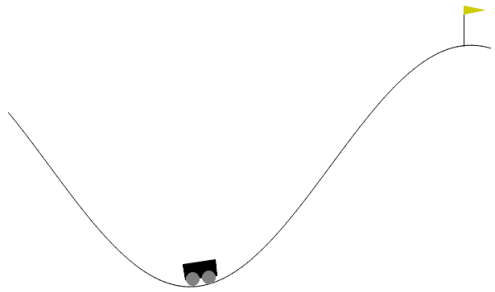
a. Thử nghiệm với môi trường "Mountain Car"

Mô tả: Sinh viên thực hiện tìm hiểu môi trường sẵn có "Mountain Car" được cung cấp bởi Gymnasium để thấy rõ cấu trúc chuẩn và vận hành của quá trình tương tác RL.

- **Bước 1:** Nhìn nhận bài toán.

Mountain Car là một trong những bài toán cổ điển nhất đại diện cho nhóm "Underactuated systems" - tức là hệ thống không đủ sức mạnh để đạt được mục tiêu một cách trực tiếp mà phải học cách tận dụng động năng.

Bài toán mô tả một chiếc xe ô tô bị kẹt dưới thung lũng giữa hai ngọn đồi. Mục tiêu là lái xe lên đỉnh ngọn đồi bên phải (vị trí đạt mức 0.5). Tuy nhiên, động cơ của chiếc xe không đủ mạnh để leo thẳng lên dốc. Để thành công, Agent phải học cách lái xe qua lại giữa hai ngọn đồi để tích lũy đủ động năng (momentum) mới có thể vọt lên đỉnh.



- **Bước 2:** Tìm hiểu về các biểu diễn Markov Decision Process (MDP) của bài toán.
  - ? *Observation Space* mô tả điều gì?
  - ? *Action Space* mô tả điều gì?
  - ? *Reward* được định nghĩa như thế nào?
  - ? Nguyên tắc hoạt động của xe?
  - ? Khi nào thì bài toán kết thúc?
- **Bước 3:** Cài đặt thư viện liên quan: `pip install -r requirements.txt`
- **Bước 4:** Mở file **Ex2.ipynb**, thực hiện nạp môi trường được cung cấp sẵn.

```
# Tạo môi trường MountainCar-v0 và khởi động nó
env = gym.make("MountainCar-v0", render_mode= "human")
# Khởi tạo episode mới với seed tái lập
observation, _ = env.reset(seed=2026)
```

- **Bước 5:** .Mở file **Ex2.ipynb**, hoàn thành các block code còn thiếu.

```
while not done and not truncated:
    # Tạo action {0,1,2} ngẫu nhiên
    action = env.action_space.sample()

    # Điều khiển Agent thực hiện action lên Environment
    new_obs, reward, done, truncated, info = env.step(action)
    total_reward += reward
    observation = new_obs
    # Hiển thị khung hình hiện tại
    env.render()

# In kết quả Reward ra màn hình và kết thúc quá trình tương tác
print("Reward: {}".format(total_reward))
env.close()
```

- **Bước 6:** Quan sát kết quả thu được.

? Hãy thử suy nghĩ hiện thực 1 giải pháp "cơ bắp" để dễ dàng đạt được mục tiêu đối với bài toán này!

b. Thử nghiệm tự tạo ra một môi trường tuỳ chỉnh "VacuumCleaner"

Mô tả: Sinh viên thực hiện tuỳ chỉnh code được cung cấp sẵn để tạo ra môi trường tuỳ chỉnh với tên gọi "VacuumCleaner". Tiến hành thực nghiệm với các chính sách khác nhau để đánh giá chính sách và môi trường.

- **Bước 1:** Phân tích bài toán và nhận định các yếu tố.

Mô tả bài toán: **VacuumCleaner** mô phỏng một robot hoạt động trong căn phòng hình lưới kích thước `mxn` với nhiệm vụ cốt lõi là dọn sạch toàn bộ bụi bẩn trong phòng. Robot được điều khiển bằng 4 hành động di chuyển cơ bản là lên, xuống, trái, phải và sẽ tự động hút sạch bụi tại ô mà nó đi qua. Để định hướng hành vi, hệ thống đưa ra các mức phần thưởng khác nhau: robot nhận điểm cộng điểm khi dọn được ô bẩn, nhưng sẽ bị trừ điểm nếu đi vào ô đã sạch hoặc va chạm vào chướng ngại vật cố định tại vị trí `(i, j)`. Phiên làm việc sẽ kết thúc ngay lập tức nếu robot đâm vào chướng ngại vật (thất bại) hoặc nhận được một khoản thưởng lớn khi đã hoàn thành việc dọn dẹp toàn bộ căn phòng (thành công).

- **Bước 2:** Cần hiểu rõ về cấu trúc của 1 Environment trong Gymnasium:

```
class VacuumCleanerEnv(gym.Env):
    def __init__(self, m=5, n=5, obstacle=(2, 2)):
        pass
        ...

    # Phương thức reset dùng để khởi tạo lại như lúc ban đầu episode đầu tiên
    def reset(self, *, seed=None, options=None):
        pass
        ...

    # Phương thức thực hiện hành đông action lên Environment
    def step(self, action):
        pass
        ...

    # Phương thức để thể hiện trực quan các kết quả, giao diện ra màn hình
    def render(self, mode='human'):
        pass
        ...
```

- **Bước 2:** Tiếp tục với file **Ex2.ipynb**, một số giải thích thêm các định nghĩa của mã code.

```
# Định nghĩa class VacuumCleanerEnv kế thừa từ gym.Env chuẩn của Gymnasium
class VacuumCleanerEnv(gym.Env):
    # Constructor của class VacuumCleanerEnv
    def __init__(self, m=5, n=5, obstacle=(2, 2)):
        super(VacuumCleanerEnv, self).__init__()
        # m và n lần lượt là chiều dài và chiều rộng để cấu thành căn phòng
        self.m = m
        self.n = n
        # obstacle là toạ độ của 1 vật cản duy nhất trong căn phòng
        self.obstacle = tuple(obstacle)
        ...
```

```
# Hàm xử lý chính của chương trình
if __name__ == "__main__":
    # Khởi tạo căn phòng có kích thước 5x5 và vật cản tại vị trí 2x2
    env = VacuumCleanerEnv(m=5, n=5, obstacle=(2, 2))
    obs, _ = env.reset()
    env.render()
    terminated = False
    truncated = False

    # Vòng lặp đến khi gặp điều kiện dừng hoặc dừng sớm
    while not terminated and not truncated:
        # Chọn ngẫu nhiên action
        action = env.action_space.sample()
        obs, reward, terminated, truncated, info = env.step(action)
        env.render()
        print(f"Action: {action}, Reward: {reward}, Terminated: {terminated}")
        if terminated or truncated:
            print("Episode finished with total reward:", env.total_reward)
            break
```

- **Bước 3:** Hoàn thành các block code được cầu tại Phần 2 trong file được cung cấp.

D. YÊU CẦU & NỘP BÀI

1. Yêu cầu

1. *Đối với Phần 1 - Tìm hiểu về Python*, sinh viên tạo tài khoản và luyện tập trên HackerRank. Bài làm phải đảm bảo số điểm đạt được **> 210 điểm trở lên**. Mỗi bài được chia

thành các section trong file **Ex1.ipynb** được cung cấp. Cung cấp về cách thức thực hiện, thuật toán và hướng giải quyết bài toán.

2. *Đối với Phần 2 - Tìm hiểu về Gymnasium*, hoàn thành các đoạn mã còn thiếu trong file **Ex2.ipynb, Phần 2**. Hiện thực chính sách **Round Robin** và **Priority-based** trên **VacuumCleaner Evironment** đã xây dựng.

<u>Gợi ý:</u>

- **Round Robin**: có thể định nghĩa robot di chuyển theo hình zic-zac (S-shape). Nó đi từ trái sang phải hết hàng 1, sau đó xuống hàng 2 và đi từ phải sang trái, cứ thế cho đến hết phòng.
- **Priority-based**: ưu tiên đi bám tường, robot sẽ đi thẳng cho đến khi chạm tường hoặc chướng ngại vật, sau đó tiến dần vào trong.

3. Thực hiện chạy với 100 episode. Viết hàm để tổng hợp Reward nhận được, vẽ biểu đồ thể hiện Reward đạt được theo thời gian. Bạn có đánh giá gì trên các giải pháp đã hiện thực và kết quả đạt được.

<u>Gợi ý:</u> sử dụng thư viện Matplotlib.

4. *Đối với Phần 3 - Hiện thực bài toán RL và đánh giá*, Hiện thực bài toán Load Balancing đơn giản với **LoadBalancingEnv**, hoàn thành các block code được yêu cầu tại file **Ex2.ipynb, Phần 3**.

5. Viết thêm hàm policy để định nghĩa các chiến lượt cân bằng tải: **Round Robin**, **Greedy** và **Epsilon-Greedy Heuristic**.

<u>Gợi ý:</u>

- **Round Robin**: chạy lần lượt từ server.
- **Greedy**: ưu tiên server có hàng đợi trống nhiều nhất để chạy.
- **Epsilon-Greedy Heuristic**: gần giống với Greedy nhưng kết hợp thêm Random để cân bằng giữa exploration và exploitation.

Pseudo Code

```
p = random()

if p < ε:
   pull random action
else:
   pull current-best action
```

6. Thực hiện chạy với 100 episode. Viết hàm để tổng hợp Reward nhận được, vẽ biểu đồ thể hiện Reward đạt được theo thời gian. Bạn có đánh giá gì trên các giải pháp đã hiện thực, kết quả đạt được, giải thích, minh chứng.

2. Yêu cầu nộp bài

- Sinh viên tìm hiểu và thực hành theo hướng dẫn. Thực hiện **theo nhóm**.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng file. Trong đó:
  - Trình bày chi tiết quá trình thực hành và trả lời các câu hỏi nếu có (kèm theo các ảnh chụp màn hình tương ứng).
  - Giải thích các kết quả đạt được.
  - Tải mẫu báo cáo thực hành và trình bày theo mẫu được cung cấp.

Nén tất cả các file và đặt tên file theo định dạng theo mẫu: **NhomY-LabX_MSSV1_MSSV2** Ví dụ: Nhom1-Lab01_29520001_29520002_29520003

  - Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.
  - Các bài nộp không tuân theo yêu cầu sẽ KHÔNG được chấm điểm.

HẾT

Chúc các bạn hoàn thành tốt bài thực hành!